

Competitive Programming

Bootcamp Day 0

ICPC Orientation, Rules, Skills, and Team Building

Ryan Pelchat

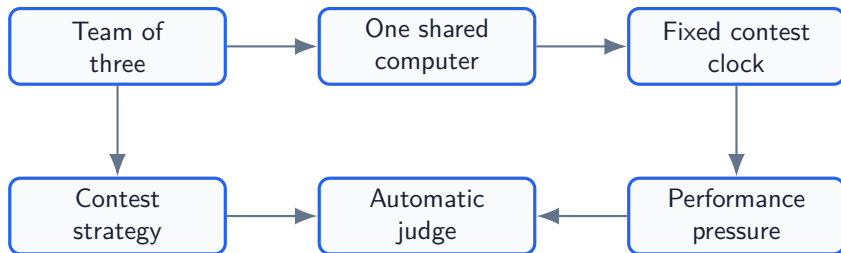
August 2026

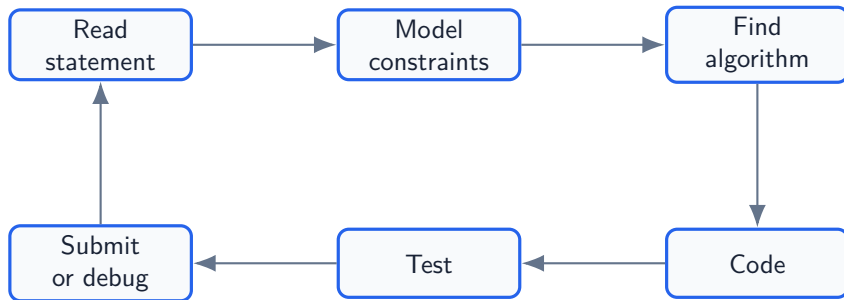
1. What ICPC is and why we train for it
2. Core ICPC-style contest rules
3. Skills needed to perform under contest pressure
4. Team building and icebreaker activity
5. Day 0 outcomes and next steps

- ▶ ICPC stands for the International Collegiate Programming Contest.
- ▶ It is a university-level, team-based programming contest.
- ▶ Teams solve algorithmic problems by writing code that is judged automatically.
- ▶ The contest rewards problem solving, programming fluency, teamwork, and calm execution.

Day 0 framing

ICPC is not only about knowing algorithms. It is about turning ideas into accepted submissions with limited time, shared equipment, and teammates depending on each other.





Team Setup

- ▶ Three contestants per team
- ▶ One shared workstation
- ▶ Communication only within the team and contest staff

Problem Set

- ▶ Several algorithmic problems
- ▶ Usually around five hours
- ▶ Submissions judged as accepted or rejected

Important

Exact eligibility, languages, printed materials, environment, and site rules are published by each official contest. Always check the current regional rules before competing.

- ▶ Teams are ranked first by the number of accepted problems.
- ▶ Ties are usually ranked by lower total penalty time.
- ▶ For each solved problem, penalty includes the accepted submission time.
- ▶ Rejected attempts before the accepted run usually add 20 minutes each.
- ▶ Unsolved problems do not add time penalty.
- ▶ Compile errors are commonly not counted as 20-minute penalties, but always confirm local rules.

Problem	Result	Attempts Before AC	Score Impact
A	Accepted at 18 min	0	18
B	Accepted at 73 min	2	$73 + 40 = 113$
C	Unsolved	5	0
Total	2 solved		131 penalty

Strategy lesson

Wrong submissions matter only when the problem is eventually solved, but repeated guessing can still waste the contest clock and slow the team.

Allowed During Contest

- ▶ Team discussion
- ▶ Approved programming environment
- ▶ Contest system submissions
- ▶ Clarification requests to judges

Generally Not Allowed

- ▶ Outside communication
- ▶ Unapproved websites or remote access
- ▶ Unauthorized electronic devices
- ▶ Tools that contact external services

When unsure

Ask a judge or proctor. In contest settings, guessing about rules is a bad risk.

Skill Area	What It Looks Like in Contest
Problem reading	Identify input, output, constraints, edge cases
Algorithm design	Match patterns to correct complexity
Implementation	Write reliable code quickly
Testing	Build small, adversarial, and boundary cases
Debugging	Isolate mistakes without panic
Strategy	Choose which problem deserves time now
Teamwork	Communicate clearly around one computer

Fluency	Algorithms	Judgment
▶ Language basics ▶ Standard library ▶ Fast input/output ▶ Templates	▶ Sorting and search ▶ Graph traversal ▶ Greedy methods ▶ Dynamic programming	▶ Complexity bounds ▶ Proof idea ▶ Edge cases ▶ Failure modes

- ▶ **Driver:** types the solution and explains implementation choices.
- ▶ **Navigator:** checks logic, watches for bugs, and keeps test cases moving.
- ▶ **Strategist:** tracks scoreboard, time, problem selection, and when to switch.

Role rule

Roles are temporary. Strong teams rotate based on the problem, not ego.

1. Everyone skims different problems.
2. Mark each problem as easy, medium, hard, or unknown.
3. Agree on the first target problem.
4. Put one person at the keyboard only after the idea is clear.
5. Keep a short list of promising next problems.

Goal

Start with a controlled solve, not three people silently staring at the same screen.

- ▶ Before coding: say the algorithm and expected complexity out loud.
- ▶ Before submitting: test sample cases, tiny cases, and boundary cases.
- ▶ After rejection: read the verdict, form one hypothesis, then test it.
- ▶ After acceptance: record the solved problem and move on quickly.

Contest habit

A fast wrong submission is rarely faster than a 60-second checklist.

1. Form groups of three.
2. Each person shares name, preferred language, strongest topic, and one topic they want to improve.
3. As a group, choose a team name.
4. Write one sentence: "We work best when..."
5. Pick initial driver, navigator, and strategist roles.

Time box

8 minutes discussion, then 1 minute share-out per team.

Task: Solve a tiny problem as a team, but only the driver may touch the keyboard.

Given a list of numbers, decide whether any two numbers sum to a target.

Example: list [2, 7, 11, 15], target 9.

Protocol

- ▶ Navigator asks questions.
- ▶ Strategist watches time.
- ▶ Driver explains code choices.
- ▶ Team rotates after 5 minutes.

- ▶ How do we decide who codes first?
- ▶ What phrase means "pause and re-check the idea"?
- ▶ When do we abandon a problem temporarily?
- ▶ How do we handle disagreement?
- ▶ What will we do after a wrong answer?

Deliverable

Each team leaves Day 0 with a short working agreement they can revise after practice contests.

- ▶ You can explain ICPC-style scoring and contest flow.
- ▶ You know the rules well enough to ask the right questions before an official contest.
- ▶ You can solve small problems using a repeatable workflow.
- ▶ Your team has an initial communication protocol.
- ▶ You start building a personal and team checklist.

ICPC rules are updated by season and by region. These slides summarize the common contest model and should be checked against the current official rules before an official event.

- ▶ ICPC Regional Rules 2026/27: <https://icpc.global/regionals/rules>
- ▶ ICPC World Finals Rules 2026: <https://icpc.global/worldfinals/rules/>
- ▶ ICPC Regionals overview: <https://icpc.global/regionals/regionals>

Welcome to the bootcamp

Learn the rules. Build the team. Solve carefully.